DLDCA Lab ?: ?

Week ?

...

Verilog Setup

We will be using `icarus-verilog` (iverilog) for simulations and VerilogHDL/SystemVerilog extension for Syntax-Highlighting and VapourView for observing the waveforms.

You can install iverilog using your package manager if you are using Linux/macOS. *For windows we recommend using WSL*, but if you insist you can check iverilog for windows.

Ubuntu / Debian / WSL

```
sudo apt update
sudo apt install iverilog
```

Fedora / RHEL

```
sudo dnf install iverilog
```

MacOS

```
brew install iverilog
```

Installation verification

Check the version of iverilog to confirm it's properly installed

```
iverilog -v
```

Running Simulations

1. **Compile**: Create an executable simulation file from your Verilog source (`design.v`) and testbench (`testbench.v`):

   ```
   iverilog -o <output-file> <input-files>
   ```

2. **Execute**: Run the simulation to generate the Value Change Dump (`.vcd`) file:

   ```
   vvp <output-file>
   ```

3. **View**: Open the resulting `.vcd` files directly in VSCode works once the extension is installed

You are given a codebase in `system-verilog` (a superset of verilog) for checking the compilation and learning to navigate the waveforms. **You are in no way expected to understand all the given code, but it can serve as a spot you can explore system-verilog.**

Quick Check

For a quick check, you can navigate to the `verilog_setup` folder and run `make` to see the following expected output:

```
verilog_setup$ make
VCD info: dumpfile build/dump.vcd opened for output.
[65000] OP=0  R0(10)  R1(20) -> R4 = 30
[75000] OP=1  R1(20)  R2(5) -> R5 = 15
[85000] OP=2  R0(10)  R3(3) -> R6 = 2
[95000] OP=3  R0(10)  R3(3) -> R7 = 11
[105000] OP=4  R4(30)  R5(15) -> R0 = 17
[115000] OP=5  R7(11)  R7(11) -> R1 = 11

Final Register File:
R0 = 17
R1 = 20
R2 = 5
R3 = 3
R4 = 30
R5 = 15
R6 = 2
R7 = 11

Simulation Complete.
tb/tb_top.sv:201: $finish called at 135000 (1ps)
```

Assembly

We will be using `x86_64` assembly standard for the duration of the course.

People using an `x86_64` processor can run this natively, but if not, you will be provided access to `mars/sl machines` where `nasm` is installed.

0.0.1 Compiling Assembly Programs

Assembly compilation is done in 2 steps:

1. Creation of object files
2. Linking of object files

```
nasm -f elf64 <file-name> -o <output-object-file>
ld <object-files> -o <executable>
```

For ease, you have been given a `Makefile`. You can just run `make` and check the expected output.

0.0.2 Logging into mars/sl-machines & Transferring Files

To connect to **mars**, use `ssh` to open a command-line interface. You can edit code directly on the remote server using an editor like `vim`, or write your code locally and transfer files back and forth using `scp` (Secure Copy Protocol).

1. SSH into the Server You can use ssh to login to mars,

```
<your-machine>$ ssh <cse-username>@mars.cse.iitb.ac.in
```

2. Transferring Files using `scp`

> **Important:** Always run `scp` commands from your **local terminal**, not from inside the active `ssh` session on `mars`.

- **Upload a local file to `mars`:**

```
scp <path-to-local-file> <cse-username>@mars.cse.iitb.ac.in:<remote
    -path>

# Example: Upload program.asm to your home directory on mars
scp program.asm <cse-username>@mars.cse.iitb.ac.in:~/
```

- **Download a file from `mars` to your local machine:**

```
scp <cse-username>@mars.cse.iitb.ac.in:<remote-file> <local-
    destination>

# Example: Download the compiled binary back to your current local
    directory (.)
scp <cse-username>@mars.cse.iitb.ac.in:~/my_executable .
```

- **Transfer an entire directory (use the `-r` recursive flag):**

```
# Upload a local project folder to mars
scp -r ./lab1_folder <cse-username>@mars.cse.iitb.ac.in:~/
```

Quick Output Check

when you run make you expect to see something like,

```
asm_setup\$ make
=== envcheck: nasm/gcc/ld pipeline test ===
[stage 1] literal write via syscall : hello from .data
[stage 2] register arithmetic (7*6+5): 47
[stage 3] conditional jump result    : branch-taken-correctly
[stage 4] loop sum 1..10             : 55
[stage 5] extern c_add(19, 23)       : 42
[stage 6] extern c_checksum(buf,8)   : 2774017321012
=== all stages completed, exit code reflects pass/fail ===
exit code: 0
```